README

Theodor Constantinescu
331CC

December 7, 2025

1 Feedback

Subiectul temei mi s-a parut interesant si practic, oferind o situatie in care paralelizarea chiar poate ajuta. Mi-a placut ca exista metode flexibile de rezolvare a cerintei si ca am invavat mai in profunzime diferite structuri de date din Java. Implementarea temei a durat in jur de 3 zile.

2 Strategia de paralelizare

Procesarea este impartita in mai multe etape distincte, fiecare fiind paralelizata in functie de numarul de thread-uri. Toate thread-urile au fost initiate in main.

Citirea si scrierea fisierelor are loc secvential, restul operatiilor au fost paralelizate in functie de numarul de thread-uri.

Etapele sunt:

1) Parsarea JSON si colectarea articolelor

Caile catre fisierele JSON sunt introduse intr-o coada de tip BlockingQueue. Fiecare thread extrage din coada si o proceseaza in paralel pana la epuizarea cozii.

Totul este retinut intr-o lista locala la nivel de thread, si se retine global printr-un ConcurrentHashMap thread-safe in care pentru fiecare titlu sau uuid e retinut numarul de aparitii.

De asemenea, numarul total de articole (cu tot cu duplicate) sunt memorate prin intermediul unei variabile de tip AtomicInteger ce se incrementeaza la fiecare articol adaugat

2) Identificarrea si eliminarea articolelor si clasificarea articolelor pe limbi si categorii

Are loc in paralel la nivelul fiecarui thread.

Dupa ce toate thread-urile termina prima parte, (sincronizata prin intermediul unei bariere CyclicBarrier), fiecare thread filtreaza in Lista globala de articole (de tip Syncronized List) doar articolele unice.

Din lista globala cu date preluate secvential din fisierele de input in main preluam categoriile si limbile mentionate. In fiecare thread, adaugam la Con-

currentHashMap un Set¡¿ de uuid-uri pentru fiecare categorie sau limbaj in parte.

La final adaug intr-o lista locala a thread-ului doar articolele in limba engleza. Asteptam pana toate thread-urile isi termina sarcina

3) Gasirea cuvintelor de interes

Preiau in fiecare thread textul articolelor in engleza din lista locala, le prelucrez conform cerintei si le pun intr-un set de cuvinte unice. La final setul este agregat unui ConcurrentHashMap global ce retine numarul total de aparitii al fiecarui cuvant

Dupa prelucrarea in paralel si apelul barierei ciclice, doar thread-ul cu ID 0 genereaza fisierul "keywordscount.txt".

4) Generarea fisierelor de iesire

Pentru limbile si categoriile preluate din input, le extrag din daca limba sau categoria respectiva exista, fac apel la perechile (title, uidds) , (categorie, uidds) umplute in etapa 2 si introduc in fisier uidds-urile ce au titlul sau categoria respectiva.

Scrierea in fisier e secventiala, fiecare thread se ocupa cu crearea unuia sau mai multor fisiere pe rand.

Dupa ce se termina crearea tuturor fisierelor, doar thread-ul cu ID 0 creaza secvential fisierul cu toate articolele, sortate lexicografic

Thread-ul cu ID 1 genereaza report.txt (avand deja toate informatiile necesare), sau cel cu ID 0 daca avem un singur thread.

Am folosit structuri de date thread-safe precum BlockingQueue pentru distribuirea dinamica a muncii intre thread-uri la parsarea fisierelor JSON si crearea fisierelor de output de tipul limbaj sau categorie

ConcurrentHashMap a fost folosit pentru toate structurile globale pentru a retine asocieri (numerice sau set de uidds) pentru limba, categorie sau cuvinte cheie.

AtomicInteger a ajutat la contorizarea globala a numarului total de articole (cu tot cu duplicate) in etapa 1

CyclicBarrier a fost folosita pentru a sincroniza toate etapele de procesare mentionate. S-a asigurat ca toate thread-urile au terminat o etapa inainte sa treaca toate la urmatoarea

Designul este corect deoarece toate structurile sunt thread-safe, iar operatiile precum incrementari sau inserari sunt atomica. Eliminarea duplicatelor si retinerea altor date (de ex. ce uidds au categoria sau limba respectiva) au fost realizate global.

Eficienta este optinuta prin distribuirea dinamica a sarcinilor folosind cozi thread-safe, minimizarea sectiunilor critice.

3 Analiza de performanta

Setup de testare: - configuratia sistemului: mediu Linux, 8CPU, RAM 16GB

- Versiune java: openjdk version "21.0.9"
- dimensiunea datasetului de test:

	Test1	Test2	Test3	Test4	Test5
Nr articole	2758	5516	8273	11031	13789

Table 1: Dimensiunea datasetului de test

Rezultate:

$$Speedup: \ S(p) = \frac{T(1)}{T(p)}$$

Nr. thread-uri	SpeedUp Teste									
	Test1		Test2		Test3		Test4		Test5	
	Timp (s)	Speedup	Timp (s)	Speedup	Timp (s)	Speedup	Timp (s)	Speedup	Timp (s)	Speedup
1	5.65	1.00	10.86	1.00	16.37	1.00	20.51	1.00	25.07	1.00
2	3.66	1.54	6.56	1.65	9.33	1.75	11.66	1.75	14.03	1.78
4	2.67	2.11	4.42	2.45	5.87	2.78	7.34	2.79	8.84	2.83

Table 2: Timp de execuție și speedup pentru teste

$$Eficienta: \ E(p) = \frac{S(p)}{p}$$

Nr. thread-uri	Eficienta Teste									
	Test1		Test2		Test3		Test4		Test5	
	Speedup	Eficienta	Speedup	Eficienta	Speedup	Eficienta	Speedup	Eficienta	Speedup	Eficienta
1	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
2	1.54	0.77	1.65	0.83	1.75	0.88	1.75	0.88	1.78	0.89
4	2.11	0.53	2.45	0.61	2.78	0.70	2.79	0.70	2.83	0.71

Table 3: Speed și eficienta pentru teste

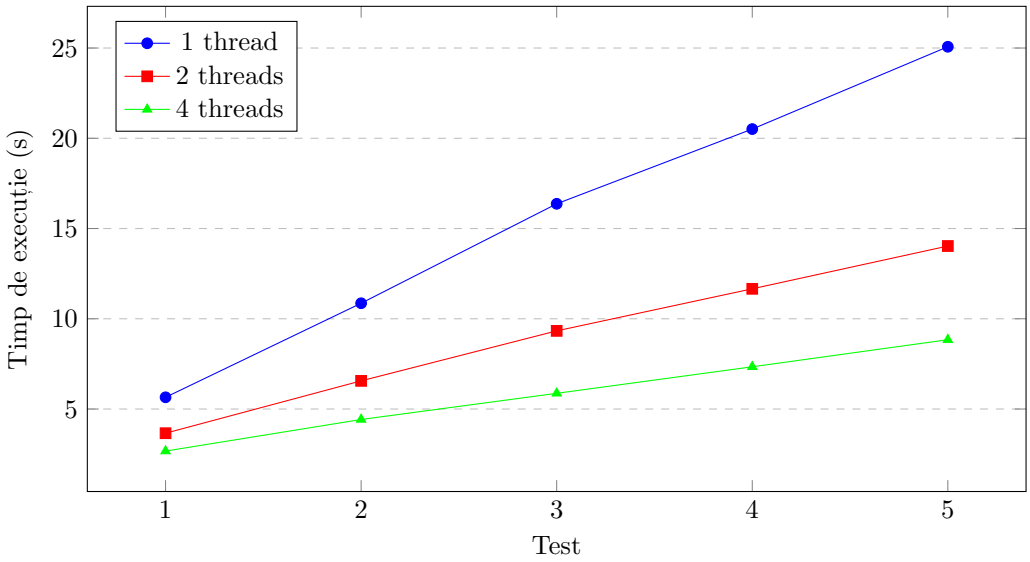


Figure 1: Timpul de execuție pe test pentru 1, 2 și 4 thread-uri

Nr. thread-uri	Rulari								
	Rulare 1			Rulare 2			Rulare 3		
	Timp (s)	Speedup	Eficiență	Timp (s)	Speedup	Eficiență	Timp (s)	Speedup	Eficiență
1	3.89	1.00	1.00	3.97	1.00	1.00	4.00	1.00	1.00
2	2.75	1.41	0.71	2.92	1.36	0.68	2.776	1.44	0.72
3	2.676	1.45	0.48	2.616	1.52	0.51	2.383	1.68	0.56
4	2.423	1.61	0.40	2.403	1.65	0.41	2.543	1.57	0.39
5	2.440	1.60	0.32	2.360	1.68	0.34	2.683	1.49	0.30

Table 4: Rularea individuala pentru datasetul 1 cu 1,2,3,4,5 threaduri

Nr. thread-uri	Timp mediu (s)	Speedup mediu	Eficiență medie
1	3.95	1.00	1.00
2	2.82	1.40	0.70
3	2.56	1.55	0.52
4	2.46	1.61	0.40
5	2.49	1.59	0.32

Table 5: Timp mediu, speedup mediu și eficiență medie pentru dataset 5 cu 1-5 thread-uri

4 Concluzii

Se observa ca timpul mediu scade odata cu cresterea numarului de thread-uri.

De asemenea speedup-ul creste aproape proportional in aproape toate testele.

La rularea individuala pentru datasetul 1 se observa ca pentru al 5-lea thread speedup-ul scade din nou, desi eficienta medie scade in continuare. Asta sugereaza faptul ca adaugarea de thread-uri suplimentare nu inseamna eficienta.

Limitarile sunt cauzate de overhead de sincronizare cauzat de tipuri de date precum ConcurrentHashMap sau synchronized List. Un numar mare de thread-uri poate anula castigul paralelizarii.

De asemenea e importanta dimensiunea datasetului, pentru un numar mai mic costul de gestionare al thread-urilor poate fi mai mare decat avantajele paralelizatii.

Conform datelor, 4 thread-uri este potrivit pentru sistemul implementat, pe cand un numar mai mare nu aduce castiguri semnificative, poate chiar diminua din speedup.